Software Engineering

Chapter 5: Software Design, Architecture & OO Principles

From Universal Foundations & Architecture Modeling to SOLID Craftsmanship

Prof. Nien-Lin Hsueh
Department of Information Engineering and Computer Science
Feng Chia University

Chapter 5: Roadmap & Core Curriculum

Part 1: Foundations & Architecture

- **5.1 Software Design Foundations:** From requirements to blueprints, why design matters, the 4 design levels, and learning roadmap.
- **5.2 Universal Design Principles:** Modularity, abstraction, information hiding, levels of cohesion & coupling.
- **5.3 Classic Architectural Patterns & Trade-offs:** Layered, repository, client-server, pipe-and-filter, MVC, microservices, and ATAM.
- **5.4 Architecture Derivation & Modeling:** Beyond patterns: ASRs, decomposition, UML vs. C4 model, 4+1 views, and ADRs.

Part 2: Detailed Design, AI & Synthesis

- **5.5 SOLID Object-Oriented Principles:** SRP, OCP, LSP, ISP, and DIP with real-world refactoring examples.
- **5.6 AI in Architecture & Design:** AI copilots for trade-off analysis, refactoring, code smell detection, and blind spot mitigations.
- **5.7 Conceptual Synthesis & References:** Core principles review, multi-dimensional self-assessment, and literature.

5.1 Software Design Foundations & Spectrum

> *"There are two ways of constructing a software design: One way is to make it so simple that there are obviously no deficiencies, and the other way is to make it so complicated that there are no obvious deficiencies."*
> *— C. A. R. Hoare (Turing Award Laureate)*

What is Software Design?

- **From Requirements (What) to Implementation (How):**
  - **Requirements Engineering**: Defines stakeholder problems and functional expectations (*what to build*).
  - **Software Design**: Conceptualizes structural decomposition, component boundaries, data models, and protocols (*how to build*).
- **Core Activities of Software Design:**
  - **Blueprint Creation**: Establishes models, abstractions, and specifications that development teams execute against.
  - **Trade-off Decision Making**: Balances competing non-functional forces (e.g., performance vs. security vs. development time and budget).
- **The Distinction Between Design and Coding:**
  - Coding expresses logic in a specific programming language syntax.
  - Design determines **how modules are partitioned, where state lives, and how components are decoupled** for long-term evolvability.

Why is Software Design Crucial?

- **1. Eliminating "Vibe Coding" and Structural Code Rot:**
  - Coding without design rapidly triggers the 4 Symptoms of Poor Design (Robert C. Martin):
    - **Rigidity**: A single change forces a cascade of modifications across dozens of unrelated files.
    - **Fragility**: Fixing a bug in Module A inexplicably breaks Module B in an unrelated subsystem.
    - **Immobility**: Components are tangled together, making reuse in other contexts impossible.
- **2. Quality Attributes are Designed In, Not Tested In:**
  - Scalability, fault tolerance, low latency, and security must be architected from day one —they cannot be patched in via unit tests later.
- **3. Barry Boehm's Cost of Defect Removal Curve:**
  - Catching an architectural flaw in **Requirements & Design** costs **$1**; in integration testing it rises to **$10**; in **production maintenance**, it escalates to **$50–$100x**!
- **4. Shared Contract & Common Language for Engineering Teams:**
  - Clean component boundaries and public interfaces enable dozens of engineers to work in parallel without blocking each other.

What Does Software Design Encompass? (Design Spectrum)

- **Software design spans a multi-level engineering spectrum from macro to micro:**
  - 🏛️ **1. Architectural Design (Macro-Structure):**
    - High-level system structure, major sub-systems, distributed communication mechanisms, and global patterns (Layered, MVC, Microservices).
  - 🔌 **2. Interface Design (Contract & Boundaries):**
    - Public API specifications, network protocols, inter-component contracts, and information hiding boundaries (encapsulating internals).
  - 🧩 **3. Component & Detailed OO Design (Micro-Implementation):**
    - Internal class structures, algorithms, data structures, SOLID object-oriented principles, and GoF design patterns.
  - 💾 **4. Data & Database Design (Persistence):**
    - Conceptual entity models, relational schemas (ERDs), NoSQL document collections, caching strategies, and data consistency models.
- **User Interface Design (UI/UX Design):**
  - Focuses on human cognitive ergonomics, usability heuristics, and interaction flows—explored in depth in **Chapter 6**.

Roadmap: Why Universal Principles Precede Architecture

- **The Educational Dilemma: Principles First vs. Architecture First:**
  - If we teach architectural patterns first, patterns risk becoming arbitrary, memorized diagrams.
  - When you understand **Separation of Concerns**, you immediately understand *why* Layered Architecture exists.
  - When you understand **Low Coupling**, you immediately understand *why* MVC isolates Models from Views.
  - When you understand **High Cohesion**, you immediately understand *why* Microservices align to Domain Bounded Contexts.
- **Our Step-by-Step Trajectory:**
  - **Step 1: Universal Design Principles (5.2)**: Core tenets, cohesion, coupling, and levels of cohesion & coupling.
  - **Step 2: Classic Patterns & Trade-offs (5.3)**: The 6 architectural styles, monolithic vs. microservices, and ATAM.
  - **Step 3: Architecture Derivation & Modeling (5.4)**: Beyond patterns: ASRs, decomposition, UML, 4+1 views, C4 model, and ADRs.
  - **Step 4: Micro Detailed Design (5.5)**: Class-level SOLID object-oriented craftsmanship.
  - **Step 5: Modern Practice & AI (5.6 & 5.7)**: AI copilot workflows, blind spot mitigations, and synthesis.

5.2 Universal Software Design Principles

> *"Good design is easily changeable; poor design resists change at every step."*
> *— Dave Thomas (The Pragmatic Programmer)*

Universal Principles: Fractal Across Every Scale

- **Software Design is Fractal:**
  - The same fundamental principles govern software quality regardless of zoom level:
    - **System Scale (Macro)**: Microservices partition business domains into decoupled, cohesive services.
    - **Module Scale (Meso)**: Packages and libraries isolate responsibilities behind public API boundaries.
    - **Class Scale (Micro)**: Individual classes encapsulate state and maintain a single, cohesive reason to change.
- **Core Tenets Guard Against Complexity:**
  - Computer memory is vast, but human working memory is strictly limited (Miller's Law: 7 ± 2 chunks).
  - Software principles are cognitive tools designed to keep software understandable, testable, and modifiable by human engineering teams.

Core Design Tenets: Modularity, Abstraction & SoC

- **1. Abstraction :**
  - Hiding low-level implementation mechanics behind clean, high-level conceptual interfaces. Allows developers to reason about systems without mental overload.
- **2. Encapsulation & Information Hiding :**
  - *David Parnas's Principle:* Bundling state and behavior inside a module while concealing private variables. Prevents external code from introducing illegal state side-effects.
- **3. Separation of Concerns (SoC):**
  - Decomposing software into distinct features that overlap as little as possible (e.g., decoupling UI rendering logic from SQL database queries).
- **4. Modularity :**
  - Dividing systems into cohesive, independently testable, and hot-swappable units.

The Golden Rule: High Cohesion & Low Coupling

- **Cohesion :**
  - Measures how tightly focused and related the internal responsibilities of a single class or module are.
  - **Engineering Goal: High Cohesion** (A module should do one thing, and do it exceptionally well).
- **Coupling :**
  - Measures the strength of direct interdependencies between separate modules.
  - **Engineering Goal: Low Coupling** (Modules communicate via abstract contracts, remaining ignorant of internal implementation details).
- **The Universal Golden Rule:**

> ***Maximize Cohesion, Minimize Coupling!***
> *High cohesion makes modules understandable and maintainable; low coupling prevents modifications from causing cascading system ripple effects.*

Cohesion & Coupling: Bad vs. Good Design

```
// ❌ BAD: Low Cohesion & High Coupling (The Infamous "God Class")
class OrderManager {
    public void processOrder() {
        // Embeds raw MySQL database query strings
        // Calculates complex VIP customer discount mathematics
        // Concatenates raw HTML strings for customer invoices
        // Opens raw low-level TCP/SMTP socket connections to send emails
    }
}
```

```
// ✅ GOOD: High Cohesion & Low Coupling (Collaborative Decoupled Classes)
class OrderProcessor {
    private final DiscountCalculator discountCalculator;
    private final OrderRepository orderRepository;
    private final NotificationService notificationService;

    public OrderProcessor(DiscountCalculator dc, OrderRepository repo, NotificationService ns) {
        this.discountCalculator = dc;
        this.orderRepository = repo;
        this.notificationService = ns;
```

Levels and Types of Cohesion: From Weak to Strong

- **Cohesion Spectrum (Edward Yourdon & Larry Constantine):**
  - Measures how strongly related the internal elements of a single module/class are:

Level	Cohesion Type	Core Characteristics	Real-World Example	Quality Assessment
7	**Functional**	Performs exactly one single, well-defined mathematical or business task	`computeSalesTax()`, `sin(x)`	**Ideal (Highest)**
6	**Sequential**	Output from one operation serves directly as the input to the next	`readData() → parseJSON() → validate()`	**Strong**
5	**Communicational**	Operations share the same input data or produce the same output	`updateCustomerAddress()` & `printLabel()`	**Acceptable**
4	**Procedural**	Elements execute in a specific order, but operate on different data	`validateOrder()`, then `updateInventoryAudit()`	**Moderate**
3	**Temporal**	Elements are grouped purely because they execute at the same time	`systemStartup()`, `loadConfigAndLog()`	**Weak (Avoid)**
2	**Logical**	Elements perform logically similar tasks, selected by a control flag	`printReport(int format)` (PDF, HTML, CSV)	**Weak (Avoid)**
		Elements have no meaningful		

Levels and Types of Coupling: From Tight to Loose

- **Coupling Spectrum (How Dependent Are Separate Modules?):**
  - Measures the degree of direct interconnections and knowledge sharing between modules:

Level	Coupling Type	Coupling Mechanism & Risk	Real-World Example	Quality Assessment
1	**Content (Pathological)**	One module reaches directly into private internal memory/code of another	Bypassing getters to modify a private field	**Worst (Severe Bug Risk)**
2	**Common (Global)**	Multiple modules share access to global mutable variables	Reading/writing global flags in multi-threading	**High Risk (Avoid)**
3	**Control**	One module controls the internal execution flow of another via flags	Passing `mode = 3` to dictate callee branch	**Moderate Risk**
4	**Stamp (Data-Structure)**	Modules share a composite data structure, but callee needs only a field	Passing entire `Customer` to get `zipCode`	**Moderate (Acceptable)**
5	**Data**	Modules communicate exclusively via elementary, primitive arguments	`calculateArea(width, height)`	**Loosest & Best (Calls)**
6	**Message / Event**	Modules communicate asynchronously via messages/events without coupling	EventBus: `OrderPlacedEvent` via Kafka	**Ideal (Decoupled)**

Concept Check Question 1 (CCQ 1)

In software system design, what is the primary practical benefit of combining **High Cohesion** with **Low Coupling**?

- **A.** Maximizes raw CPU runtime performance by inlining all functions into a single binary.
- **B.** Produces a modular architecture where changes to one class cause minimal cascading ripple effects on other modules.
- **C.** Mandates storing all central data structures in a globally shared variable space.
- **D.** Completely eliminates the need to write automated integration tests and regression suites.



5.3 Classic Architectural Patterns & Trade-offs

> *"There are no solutions, only trade-offs."*
> *— Thomas Sowell*

What is Software Architecture?

- **Formal Definition (IEEE 1471 / ISO 42010):**

> *Software architecture is the fundamental organization of a system embodied in its components, their relationships to each other and the environment, and the principles guiding its design and evolution.*

- **The Bridge Role of Architecture:**
  - **Requirements (What)**: Stakeholder needs, user stories, and constraints.
  - **Implementation (How)**: Source code, data schemas, and unit tests.
  - **Architecture (The Bridge)**: Transforms messy requirements into structured sub-systems, communication boundaries, and deployment topology.
- **Architecture in the Small vs. in the Large:**
  - **In the Small**: Internal design of individual applications (classes, modules, local interfaces).
  - **In the Large**: Enterprise-scale structures (distributed services, cloud platforms, event buses, multi-tier databases).

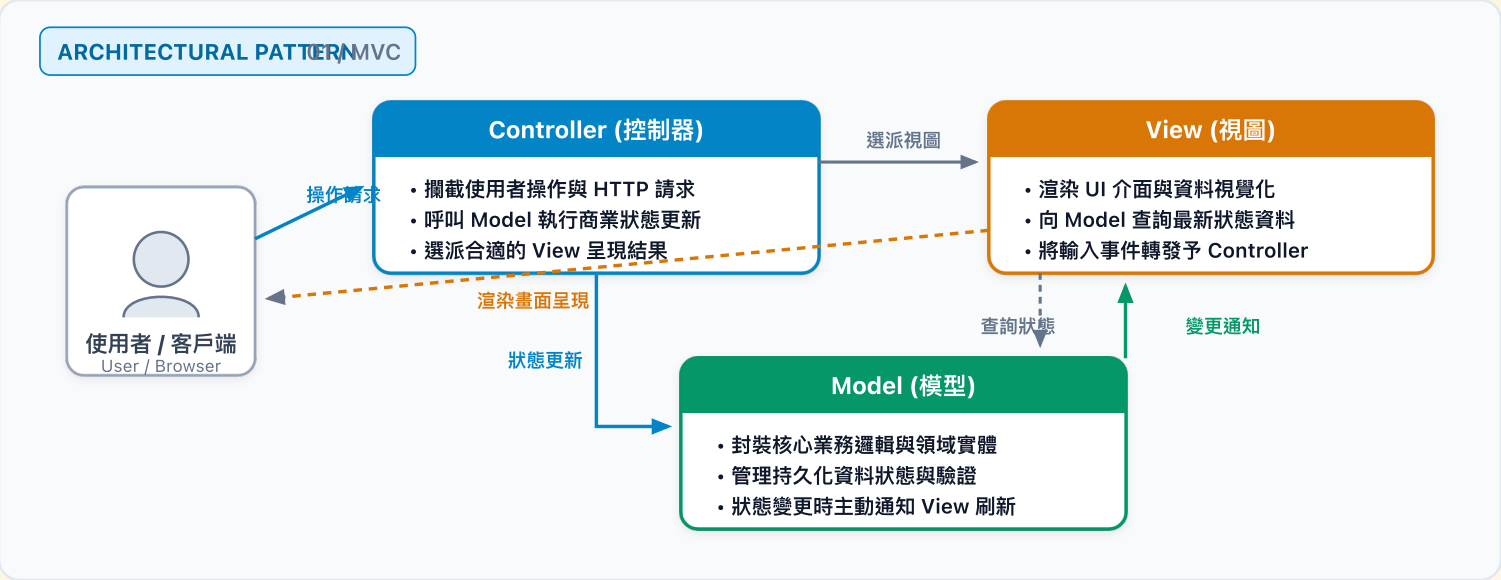
What is an Architectural Pattern?

- **Definition of Architectural Patterns:**
  - An architectural pattern is a **reusable, established solution to a commonly occurring structural problem** in software system design.
  - While design patterns (e.g., Factory, Strategy) operate at the class level, architectural patterns govern the **entire macro-system topology**.
- **Three Key Elements of an Architectural Pattern:**
  - **1. Context**: The recurring design situation and engineering environment.
  - **2. Problem & Forces**: Competing requirements (e.g., decoupling UI from business logic vs. minimizing latency).
  - **3. Solution & Consequences**: Structural partitioning, communication topology, and resulting trade-offs.
- **Why Patterns Matter:**
  - Establishes a shared architectural vocabulary across engineering teams.
  - Codifies decades of engineering experience, avoiding recurring structural failures.

Architectural Patterns: Comprehensive Overview Matrix

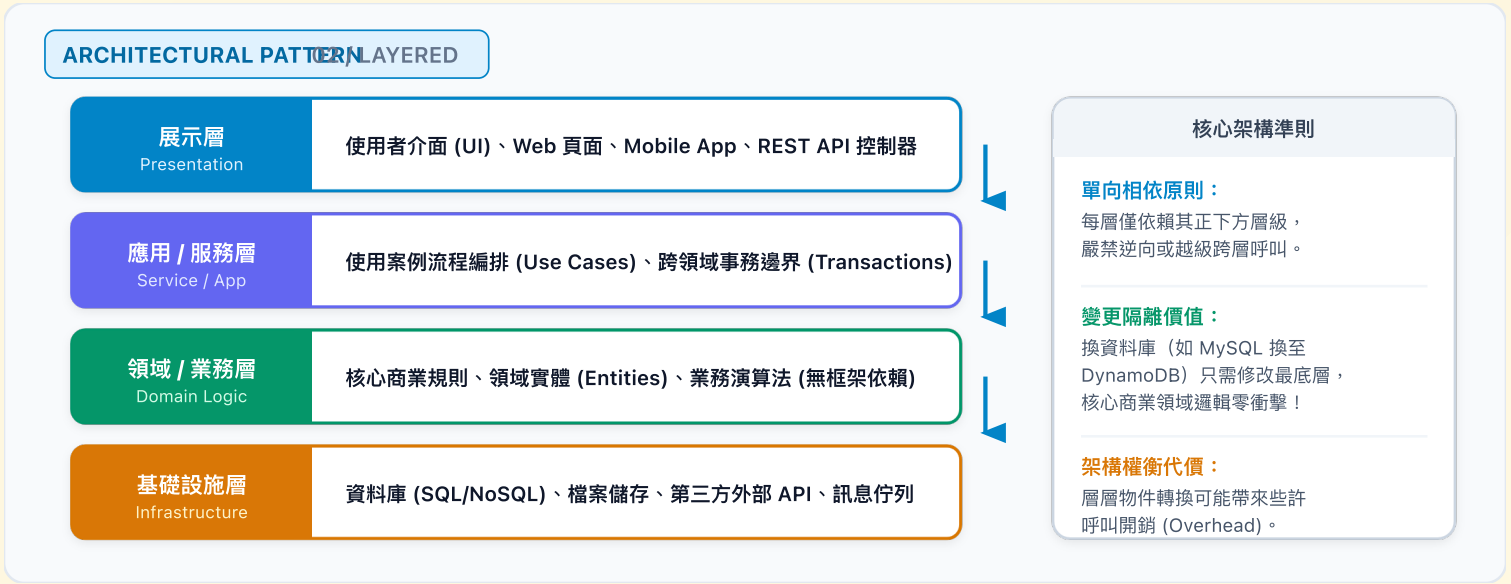
Pattern Name	Visual Topology	Core Mechanism	Best Used For	Primary Risk & Trade-off
1. MVC	Triangular Cycle (`C → M → V`)	Separates UI presentation from data models	Interactive Web / mobile apps with rich UIs	Overkill for simple data display (CRUD)
2. Layered	Vertical Stack (`N → N-1`)	Strict hierarchy with one-way downward calls	Enterprise information systems & CRUD apps	Performance overhead; sinkhole anti-pattern
3. Repository	Hub-and-Spoke Central Store	Passive central data store with active agents	Compilers, IDEs, and shared data analysis	Central DB bottleneck & single point of failure
4. Client-Server	Distributed Network (`C ↔ S`)	Clients request services from central servers	Web apps, file servers, centralized services	Network latency & server overloading
5. Pipe-Filter	Linear Sequential Stream	Discrete transform filters connected via pipes	Data processing, compilers, ETL pipelines	Inefficient for interactive systems; parsing cost
6. Microservices	Decentralized Mesh (`S ↔ S`)	Independently deployable domain services	Large-scale, rapidly evolving cloud platforms	High operational, network & transactional complexity

1. Model-View-Controller (MVC) Pattern



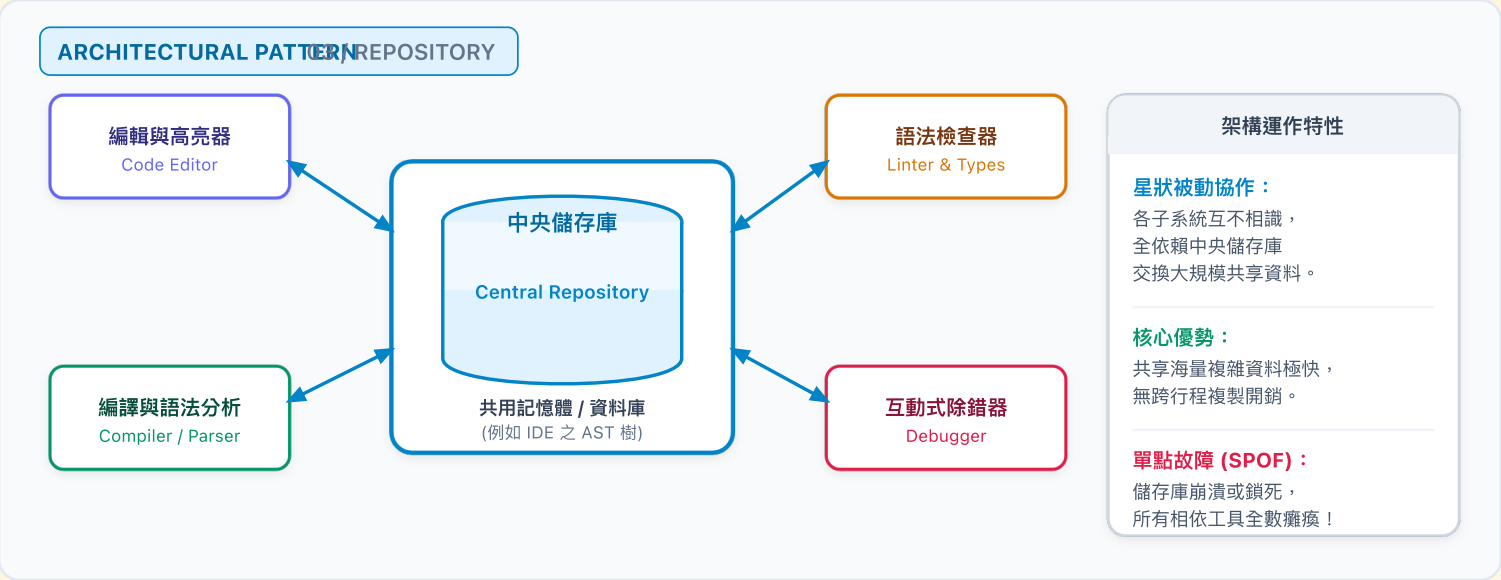
- **Core Structural Responsibilities:**
  - **Model**: Encapsulates core business data, validation logic, and domain state.
  - **View**: Renders visual representation to users; listens for state change events.
  - **Controller**: Intercepts user inputs, orchestrates business logic, and mutates the Model.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Enables multiple simultaneous views of the same model; allows UI and backend engineers to work in parallel.
  - **Weaknesses**: Increases complexity for simple screens; tight coupling can occur if Views bypass Controllers.

2. Layered Architecture Pattern



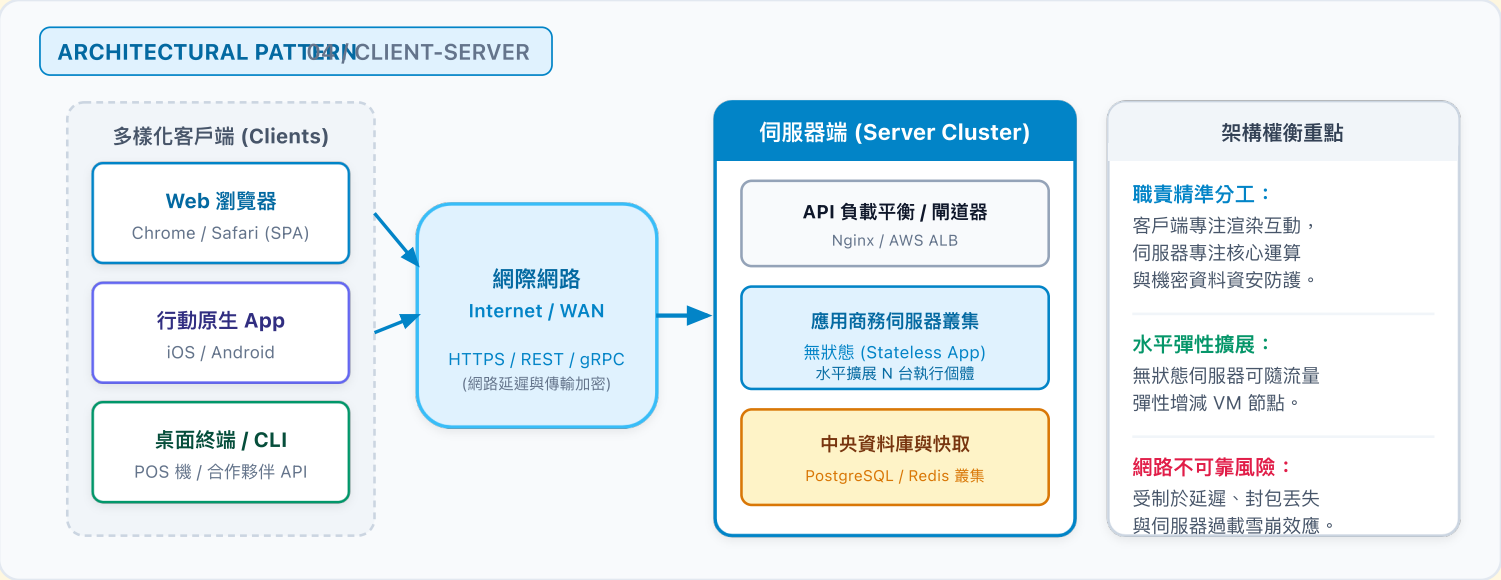
- **Core Structural Responsibilities:**
  - Organizes components into horizontal layers: **Presentation → Business Logic → Data Access → Database**.
  - **Strict Layering Rule**: Each layer only depends on the layer immediately below it (`Layer N → Layer N-1`).
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: High modularity; hot-swappable layers (e.g., swapping SQL for NoSQL without modifying business logic).
  - **Weaknesses**: "Architecture Sinkhole Anti-pattern" (requests simply pass through layers

3. Repository Architecture Pattern



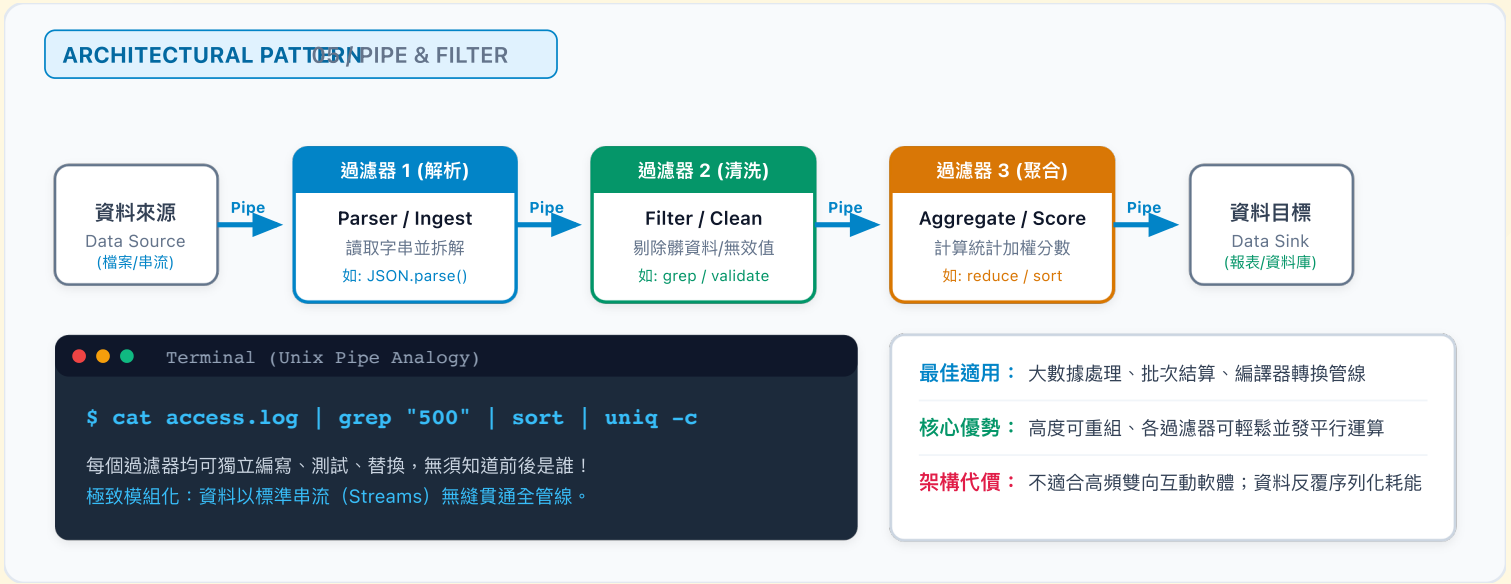
- **Core Structural Responsibilities:**
  - **Central Repository**: Passive data store maintaining all persistent system state.
  - **Subsystems / Accessors**: Independent active components that interact solely through the repository.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Efficient sharing of large, complex data structures; subsystems do not need direct inter-connections.
  - **Weaknesses**: Repository becomes a single point of failure (SPOF) and a severe performance bottleneck under high concurrent writes.

4. Client-Server Architecture Pattern



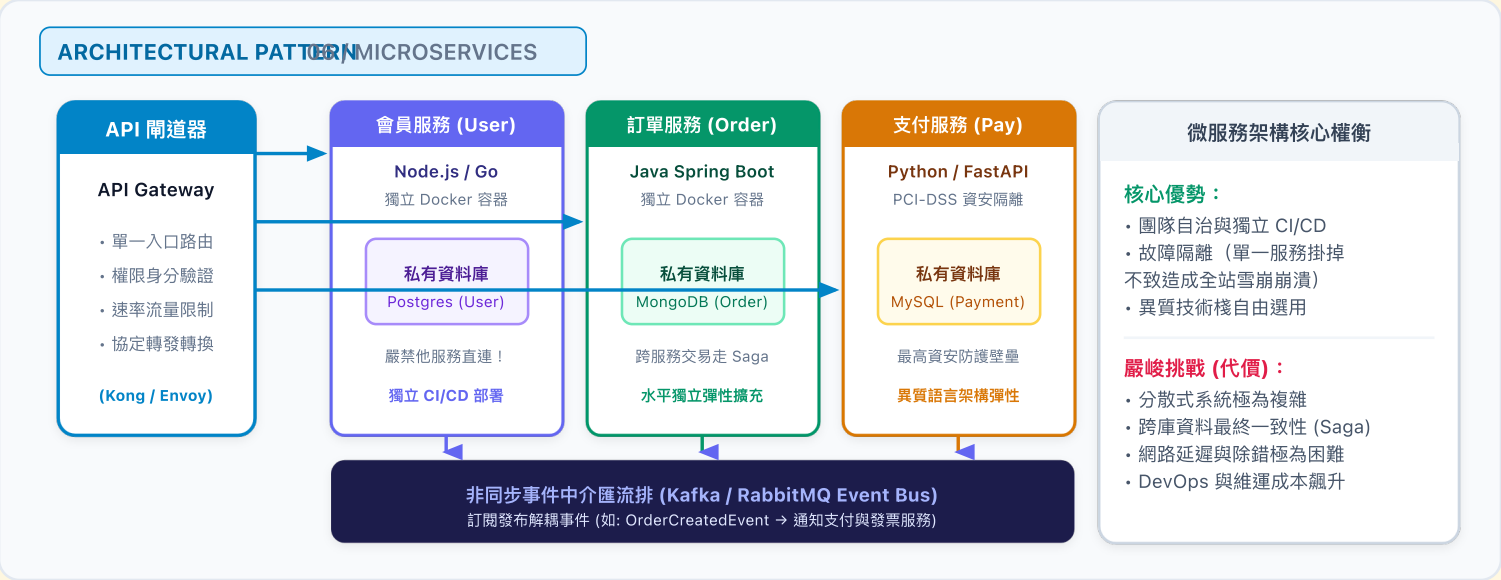
- **Core Structural Responsibilities:**
  - **Server Component**: Centralized provider offering shared services (data storage, compute, auth).
  - **Client Components**: Distributed consumers that initiate network requests to access server resources.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Centralized data management, access control, and seamless server-side upgrades.
  - **Weaknesses**: Network latency and failure risks; server overload during traffic surges

5. Pipe-and-Filter Architecture Pattern



- **Core Structural Responsibilities:**
  - **Filters**: Independent, stateless transformation components that read input streams and produce output streams.
  - **Pipes**: Unidirectional channels that buffer and transmit data streams between filters.
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: High reusability (filters can be recombined freely, like Unix pipes `cat | grep | sort`); easy parallelization.
  - **Weaknesses**: Poor fit for interactive GUI systems; data parsing overhead between adjacent filter boundaries.

6. Microservices Architecture Pattern (MSA)



- **Core Structural Responsibilities:**
  - Decomposes the application into a suite of small, autonomous, **independently deployable services**.
  - **Database per Service**: Each microservice owns its private persistent database; direct cross-service DB queries are forbidden!
- **Engineering Advantages & Trade-offs:**
  - **Strengths**: Independent deployment cadence; polyglot technology stacks; strict fault isolation (blast radius containment).
  - **Weaknesses**: Distributed systems complexity: network latency, distributed transactions

Monolithic vs. Microservices: Deep Dive

Comparison Dimension	Monolithic Architecture	Microservices Architecture
Deployment Unit	Single unified binary / WAR / container	Dozens/hundreds of independent containers
Data Storage	Single shared relational database	**Database-per-Service** (strict data encapsulation)
Communication	Fast in-memory function calls	Network RPC / REST / gRPC / Event Bus
Fault Isolation	**Poor**: Memory leak in Module A crashes entire app	**Superior**: Payment service crash leaves Browse active
Team Scalability	Merge conflicts; coordination bottlenecks	Independent feature teams deploying autonomously
Operational Cost	Simple (single CI/CD pipeline, standard logs)	High (Kubernetes, distributed tracing, service mesh)
Architectural Sweet Spot	Startups, MVP stage, small teams (<15 devs)	Large organizations (>50 devs), complex domains

Architecture Directly Shapes Quality Attributes

- **Quality Attributes Cannot Be Retrofitted Later:**
  - You cannot build a tightly coupled monolith and hope a few unit tests will magically make it scalable.
- **How Architectural Decisions Determine Quality Attributes:**
  - ⚡ **Performance**: Dictated by network hops, in-memory caching layers, and database partitioning.
  - 🛡️ **Security**: Dictated by zero-trust network zones, API gateways, and isolation of sensitive databases.
  - 🦺 **Safety**: Dictated by structural isolation of safety-critical functions (e.g., flight control vs. in-flight entertainment).
  - 🔄 **Availability & Fault Tolerance**: Dictated by redundancy, health checks, circuit breakers, and automated failover replicas.
  - 🔧 **Maintainability & Modifiability**: Dictated by strict adherence to High Cohesion and Low Coupling boundaries.

Architectural Trade-off Analysis & ATAM

- **The Inevitable Conflict Matrix:**
  - In software architecture, there are **no perfect solutions—only trade-offs**:
    - **Performance vs. Security**: Deep packet encryption and multi-tier authentication add latency.
    - **Availability vs. Consistency (CAP Theorem)**: Instant consistency requires blocking network locks, hurting availability.
    - **Modularity vs. Performance**: Pure decoupled layers introduce serialization and call-stack overhead.
- **ATAM (Architecture Trade-off Analysis Method):**
  - Developed by Carnegie Mellon SEI (Software Engineering Institute) to evaluate architectural soundness before coding:
  - **Key Concepts in ATAM:**
    - **Sensitivity Points**: Architectural decisions directly affecting a single quality attribute.
    - **Trade-off Points**: Architectural decisions affecting multiple competing attributes (e.g., encryption improves security but hurts throughput).
    - **Risks & Non-Risks**: Decisions that threaten or safeguard system viability.

Concept Check Question 2 (CCQ 2)

When comparing a Monolithic architecture with a Microservices architecture, what is the primary structural reason that Microservices provide superior **fault isolation** (blast radius containment)?

- **A.** Microservices store all enterprise data inside a single shared database instance.
- **B.** Each microservice executes in an isolated process with its own private database, preventing catastrophic cascading failures.
- **C.** Microservices completely eliminate all network communication overhead and distributed latency.
- **D.** Microservices require significantly fewer software engineers to deploy and operate.



5.4 Architecture Derivation & Modeling: Beyond Patterns

> *"Architecture is the decisions that you wish you could get right the first time."*
> *— Ralph Johnson (Design Patterns Author)*

How to Do Architecture Design: Beyond Just Picking Patterns

- **Architecture is NOT a "Pattern Shopping Cart":**
  - Architecture is a disciplined engineering process of discovery, partitioning, and trade-off synthesis:
- **The 5-Step Architecture Derivation Process:**
  - **1. Extract ASRs (Architecturally Significant Requirements)**:
    - Identify high-priority Quality Attributes (e.g., 99.99% uptime, <100ms latency) and mission-critical Use Cases.
  - **2. Subsystem Partitioning & Decomposition**:
    - Decompose the system by business capabilities (Domain-Driven Bounded Contexts) or technical layers.
  - **3. Synthesize & Combine Architectural Styles**:
    - Real-world systems combine patterns: e.g., Microservices at the enterprise tier, Event-Driven for ingestion, and Layered MVC inside each service!
  - **4. Define Communication Protocols & Interfaces**:
    - Synchronous REST / gRPC vs. asynchronous Kafka / RabbitMQ event streams.
  - **5. Evaluate Trade-offs & Document Decisions**:
    - Validate resilience under stress (ATAM) and capture decisions in Architecture Decision Records (ADRs).

Modeling Architecture: UML in Practice vs. Common Pitfalls

- **The "Class Diagram" Trap (Most Common Novice Mistake):**
  - Beginners often draw a giant UML Class Diagram with 50 classes and call it "System Architecture."
  - **Why this fails:** Class diagrams show compile-time object relationships; they completely conceal network boundaries, runtime processes, database servers, and cloud deployment topology!
- **Traditional UML Diagrams for Macro-Architecture:**
  - **1. Component Diagram**:
    - Visualizes high-level software components, provided interfaces (`--○` lollipop), and required interfaces (`--(` socket).
  - **2. Package Diagram**:
    - Depicts namespace organization, logical subsystem boundaries, and dependency arrows.
  - **3. Deployment Diagram**:
    - Models physical hardware nodes, virtual containers, execution environments, and network protocols (HTTPS, TCP/IP, JDBC).

Multi-Perspective Architecture: Kruchten's 4+1 View Model

- **Philippe Kruchten (1995): "No single diagram can capture architecture for all stakeholders."**
- **The 4+1 Multi-Perspective Framework:**
  - **1. Logical View (End-User / Functional)**:
    - Class structures, entity models, and domain packages satisfying functional user requirements.
  - **2. Process View (System Integrators / Performance)**:
    - Concurrency, multi-threading, asynchronous message queues, IPC, and runtime performance.
  - **3. Development View (Software Developers / Managers)**:
    - Source code package hierarchy, libraries, build modules, and repository organization.
  - **4. Physical View / Deployment View (System Engineers / DevOps)**:
    - Physical servers, cloud containers, network topology, firewalls, and communication protocols.
  - **+1. Scenarios (Use Cases / Drivers)**:
    - Core business workflows that bind, validate, and animate all four views together.

Modern Architecture Visualization: The C4 Model

- **Created by Simon Brown as a modern, developer-friendly alternative to heavy UML:**
- **The "Google Maps" Metaphor for Software Architecture (4 Zoom Levels):**
  - 🗺️ **Level 1: System Context (Zoom: Continental View)**:
    - Who uses the system (Actors) and what external systems does it integrate with (e.g., Stripe, SendGrid)?
  - 🌆 **Level 2: Container Diagram (Zoom: City Street View)**:
    - What are the deployable units? (SPA Web App, iOS App, API Gateway, Microservice, DB, Redis Cache) and their protocols (JSON/HTTPS, gRPC).
  - 🏛️ **Level 3: Component Diagram (Zoom: Building Floorplan View)**:
    - Inside a single container: What are the major functional components (e.g., AuthController, PaymentService, UserRepository)?
  - 🚪 **Level 4: Code Diagram (Zoom: Room Detail View)**:
    - Class diagrams showing internal algorithms and interfaces (typically optional; auto-generated).

Documenting Architectural Decisions: Architecture Decision Records (ADRs)

- **Why ADRs are Essential:**
  - Code shows *how* a system is implemented, but code never explains *why* critical architectural choices were made.
  - Without ADRs, engineering teams suffer from **"Architectural Archaeology"**—spending weeks wondering why an unusual database or protocol was chosen.
- **Standard Structure of an ADR (Michael Nygard Template):**
  - **1. Title**: Numbered, descriptive decision (e.g., `ADR-005: Adopt Kafka for Order Event Streaming`).
  - **2. Status**: `Proposed` → `Accepted` → `Deprecated` → `Superseded by ADR-XXX`.
  - **3. Context**: The business problem, technical forces, and constraints driving the decision.
  - **4. Decision**: The concrete architectural strategy adopted and rationale.
  - **5. Consequences**: Both positive benefits gained AND negative trade-offs accepted.

Concept Check Question 3 (CCQ 3)

In Philippe Kruchten's 4+1 Architectural View Model, which view is specifically responsible for depicting how software components are deployed onto physical server hardware, cloud containers, and network topology?



- **A.** Logical View
- **B.** Process View
- **C.** Development View
- **D.** Physical View (Deployment View)

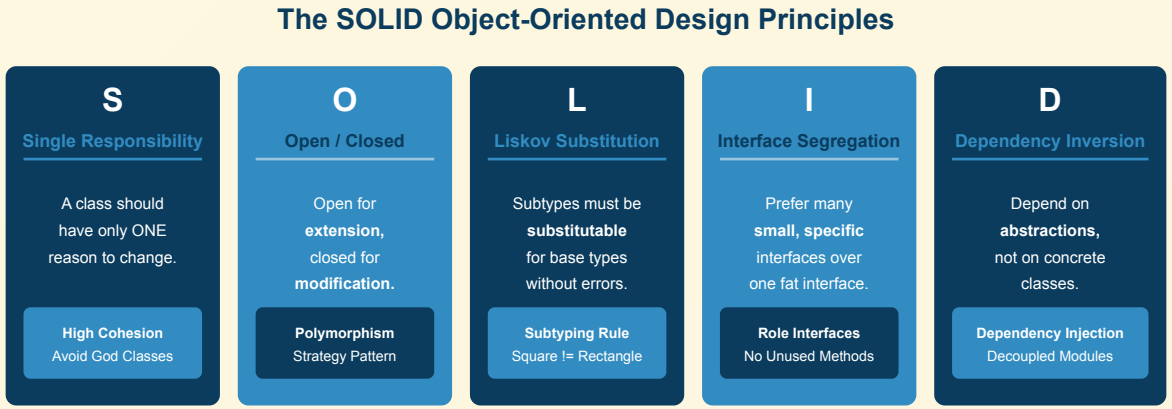
5.5 SOLID Object-Oriented Design Principles

> *"Clean code always looks like it was written by someone who cares."*
> *— Robert C. Martin (Uncle Bob)*

From Macro Architecture to Micro Detailed Design

- **Connecting the Architectural Vision to Source Code:**
  - Macro architecture establishes the structural skeleton and subsystem boundaries.
  - Micro detailed design fills that skeleton with robust, maintainable classes and methods.
- **The "Micro-Decay" Threat:**
  - Even the most elegant microservices or layered architecture will collapse into chaos if the code inside each service degenerates into tightly coupled spaghetti!
- **The Five SOLID Principles (Coined by Robert C. Martin):**
  - **S**: Single Responsibility Principle (SRP)
  - **O**: Open / Closed Principle (OCP)
  - **L**: Liskov Substitution Principle (LSP)
  - **I**: Interface Segregation Principle (ISP)
  - **D**: Dependency Inversion Principle (DIP)

Overview: The 5 SOLID Design Principles



- **The SOLID Roster:**
  - **SRP**: A class should have one, and only one, reason to change.
  - **OCP**: Software entities should be open for extension, but closed for modification.
  - **LSP**: Subtypes must be substitutable for their base types without altering correctness.
  - **ISP**: Clients should not be forced to depend on methods they do not use.
  - **DIP**: High-level policies should not depend on low-level details; both should depend on abstractions.

S – Single Responsibility Principle (SRP)

- **Core Principle:**

> *A class should have one, and only one, reason to change.*

- **Why SRP is Vital:**
  - When a class juggles multiple responsibilities, requirements changes from disparate stakeholders become tangled together, creating unexpected regressions.

```
// ❌ BAD: Multiple Reasons to Change (God Class violates SRP)
class UserInvoiceReport {
    public void calculateTotal() { /* CFO / Finance change */ }
    public void renderHtmlPage()  { /* Marketing / UI change */ }
    public void saveToDatabase()  { /* DBA / Schema change   */ }
}
```

```
// ✅ GOOD: Single Responsibility per Class
class InvoiceCalculator { public BigDecimal calculateTotal(Invoice inv) { ... } }
class InvoiceHtmlRenderer { public String render(Invoice inv) { ... } }
class InvoiceRepository { public void save(Invoice inv) { ... } }
```

O – Open / Closed Principle (OCP)

- **Core Principle (Bertrand Meyer):**

> *Software entities (classes, modules, functions) should be **open for extension**, but **closed for modification**.*
>
> ”

- **How to Achieve OCP:**
  - Program to abstract interfaces; introduce new behaviors by adding new classes, **never by modifying battle-tested existing code** with brittle `if/else` or `switch` chains!

```
// ❌ BAD: Adding a new channel forces modifying existing sendNotification code!
if (type.equals("EMAIL")) { sendEmail(); }
else if (type.equals("SMS")) { sendSms(); }
else if (type.equals("DISCORD")) { sendDiscord(); } // Risky modification!
```

```
// ✅ GOOD: Open for Extension via Polymorphic Strategy Pattern
interface NotificationChannel { void send(String message); }
class EmailChannel implements NotificationChannel { ... }
class DiscordChannel implements NotificationChannel { ... } // Zero modification to core!
```

L – Liskov Substitution Principle (LSP)

- **Core Principle (Barbara Liskov, Turing Award Laureate):**

> *Subtypes must be substitutable for their base types without altering the correctness of the program.*
>
> ”

- **The Classic Violation: The Square-Rectangle Problem:**
  - In mathematics, a Square is a Rectangle. But in software, overriding `setWidth(w)` to also mutate `height` breaks caller expectations!

```
// ❌ BAD: Violating LSP via Unexpected Side-Effects
class Rectangle {
    protected int w, h;
    public void setWidth(int w) { this.w = w; }
    public void setHeight(int h) { this.h = h; }
    public int getArea() { return w * h; }
}
class Square extends Rectangle {
    @Override public void setWidth(int w) { this.w = this.h = w; } // Unexpected side-effect!
}
```

I – Interface Segregation Principle (ISP)

- **Core Principle:**

> *Clients should not be forced to depend on methods they do not use.* „

- **Key Tenet:** Prefer multiple **small, role-specific interfaces** over a single bloated, "fat" interface.

```
// ❌ BAD: Fat, Bloated Interface forces dummy implementations
interface MultiFunctionPrinter {
    void print();
    void scan();
    void fax();
}
class SimpleInkjetPrinter implements MultiFunctionPrinter {
    public void print() { /* OK */ }
    public void scan()  { throw new UnsupportedOperationException(); } // Polluted!
    public void fax()   { throw new UnsupportedOperationException(); } // Polluted!
}
```

```
// ✅ GOOD: Segregated, Role-Specific Interfaces
```

D – Dependency Inversion Principle (DIP)

- **Core Principle:**

> 1. *High-level modules should not depend on low-level modules; both should depend on abstractions.*
> 2. *Abstractions should not depend on details; details must depend on abstractions.*
>
> ”

```
// ❌ BAD: High-level business logic hardwired to low-level MySQL database
class OrderService {
    private MySQLDatabase db = new MySQLDatabase(); // Direct 'new' coupling!
}
```

```
// ✅ GOOD: Inverted Dependency via Constructor Injection (IoC / DI)
interface Database { void save(Order o); }
class MySQLDatabase implements Database { ... }
class MockDatabase implements Database { ... } // Fast in-memory unit tests!

class OrderService {
```

Concept Check Question 4 (CCQ 4)

Which SOLID object-oriented design principle explicitly advocates that when introducing new system capabilities, developers should extend functionality by adding new classes rather than modifying existing, battle-tested code?

- **A.** Single Responsibility Principle (SRP)
- **B.** Open / Closed Principle (OCP)
- **C.** Interface Segregation Principle (ISP)
- **D.** Liskov Substitution Principle (LSP)



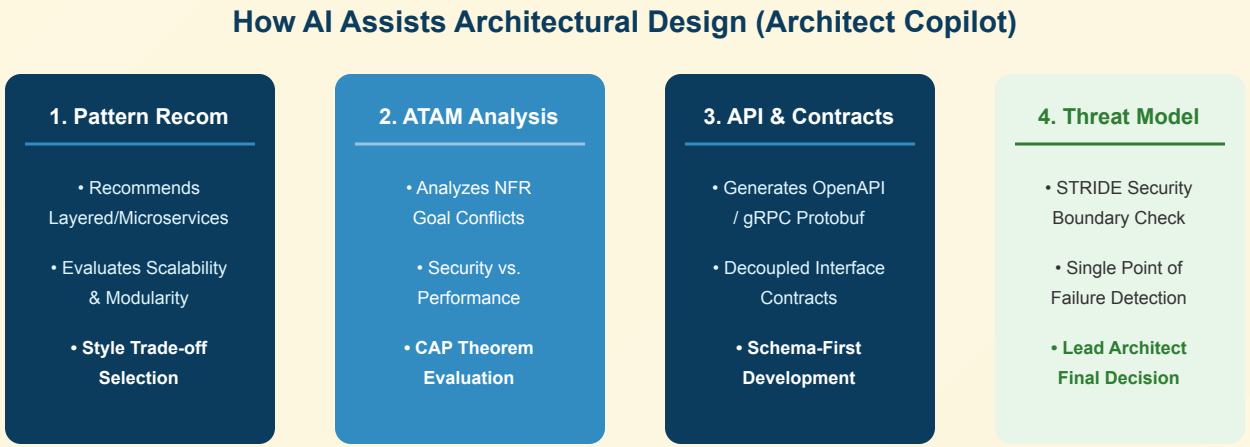
5.6 AI in Software Design & Architecture

> *"AI will not replace software architects, but architects who use AI will replace those who do not."*

AI in Architecture & Design: The Copilot Paradigm

- **The Evolution of the Software Architect's Toolkit:**
  - **Traditional Workflow**: Manual whiteboard diagramming, static architecture reviews, and labor-intensive manual refactoring.
  - **AI-Augmented Workflow**: Conversational trade-off exploration, automated architectural smell scanning, and prompt-driven interface generation.
- **The "Copilot, Not Autopilot" Principle:**
  - AI excels at **generating options, synthesizing documentation, and identifying edge cases**.
  - Human architects must provide **organizational context, budget constraints, security governance, and final accountability**.

4 Core Applications of AI in Software Design



- **1. Architectural Trade-off Exploration (ATAM Support):**
  - Prompting LLMs to simulate adversarial scenarios: "What happens to our payment pipeline if Redis crashes during a network partition?"
- **2. Automated Code Smell & Anti-Pattern Detection:**
  - Scanning codebases for God Classes, Feature Envy, circular dependencies, and SRP violations.
- **3. Design Pattern & SOLID Refactoring Suggestions:**
  - Transforming tight `switch/case` statements into polymorphic Strategy or Factory patterns.

Human-in-the-Loop: Mitigating AI Blind Spots

- **Risks of Unchecked AI Adoption:**
  - **Premature Distributed Complexity:** AI frequently over-engineers, recommending microservices, Kubernetes, and Kafka for simple internal tools that 3 engineers could finish in a fortnight with a monolith.
  - **Context Blindness:** AI is blind to your team's tech-stack familiarity, cloud hosting budgets, and internal security compliance constraints.
  - **Superficial Refactoring:** Renaming variables without resolving fundamental architectural bottlenecks or data coupling.
- **The Golden Rule of System Architecture:**

> ***"AI Proposes Trade-offs; Lead Architects Make Decisions!"***
> *Human engineers must evaluate organizational context and bear ultimate responsibility for system viability.*

5.7 Conceptual Synthesis & References

> *"Simplicity is prerequisite for reliability."*
> *— Edsger W. Dijkstra*

Core Curriculum Self-Assessment & Key Takeaways

Curriculum Dimension	Foundational Concept	Golden Architectural Heuristic
5.1 Design Spectrum	4 Design Levels	Catch architectural flaws early; design quality in from day one.
5.2 Universal Principles	Cohesion & Coupling	**Maximize Cohesion, Minimize Coupling!** Limit blast radiuses.
5.3 Architecture Modeling	4+1 Views & C4 Model	Beyond patterns: derive architecture from ASRs; document in ADRs.
5.4 Classic Patterns	6 Structural Archetypes	There are no perfect solutions, only evaluated trade-offs (ATAM).
5.5 Detailed OO Design	5 SOLID Principles	Depend on abstractions (DIP); open for extension, closed for modification (OCP).
5.6 AI in Design	Copilot Paradigm	AI evaluates code; human architects evaluate organizational context!

Classic Literature & Essential References

- **Foundational Software Architecture Textbooks:**
  - Len Bass, Paul Clements, Rick Kazman (SEI): *Software Architecture in Practice*, 4th Edition, Addison-Wesley.
  - Philippe Kruchten: "*Architectural Blueprints—The 4+1 View Model of Software Architecture*", IEEE Software, 1995.
  - Simon Brown: *Software Architecture for Developers (The C4 Model)*, Leanpub.
- **Classic Design Principles & Patterns Literature:**
  - Robert C. Martin (Uncle Bob): *Clean Architecture: A Craftsman's Guide to Software Structure and Design*, Prentice Hall.
  - Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides: *Design Patterns: Elements of Reusable Object-Oriented Software (GoF)*, Addison-Wesley.
  - Martin Fowler: *Patterns of Enterprise Application Architecture (P of EAA)*, Addison-Wesley.
  - Michael Nygard: *Documenting Architecture Decisions (ADR)*, 2011.

Thank You for Listening!

Questions & Discussion

Prof. Nien-Lin Hsueh
Department of Information Engineering and Computer Science
Feng Chia University